Algorithms

Week 9 – Search Trees

Korea University Sejong Campus, Dept. of Computer Science & Software

Part 1. Tree Fundamentals and Binary Search Trees

Learning Objectives

- Understand the **recursive structure** of trees and tree traversal complexity
- Analyze **BST** operations: search, insert, delete
- Recognize BST **average-case O(log n)** and **worst-case O(n)** behavior
- Understand **Red-Black Trees** and guaranteed O(log n) height
- Learn **B-Trees** and disk I/O optimization through wide, shallow structure

Textbook: CLRS 3rd Edition, Chapters 12–13

Recursive Definition of a Tree

A **tree** is a root node with zero or more **subtrees** — a recursive structure.

```
          [A]                  "A tree of n nodes
         / | \                  = root + subtrees"
       /   |   \
     [B]  [C]  [D]            Each subtree is itself
     / \       / | \          a smaller tree.
   [E] [F]   [G][H][I]
```

Key insight: The "repetition of partial structure" lets us implement tree algorithms with **recursion**.

- This is the same principle behind **divide and conquer**
- Split the problem into subproblems (subtrees), solve each, combine results

Tree Traversal Complexity

For a **balanced** binary tree with n nodes, traversal visits every node exactly once:

$$T(n) = 2T(n/2) + O(1)$$

- **2T(n/2)**: Traverse left and right subtrees (each has ~n/2 nodes)
- **O(1)**: Visit the root node

Applying the Master Theorem: a = 2, b = 2, f(n) = O(1)

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Since f(n) = O(1) = O(n^0), and 0 < 1, this is **Case 1**:

$$T(n) = \Theta(n)$$

Tree traversal is **linear** in the number of nodes. This makes sense – we visit each node exactly once, so we cannot do better than O(n).

Binary Search Tree (BST) – The Idea

A BST is a binary tree that maintains a **sorted order**:

$$\text{left subtree} < \text{root} < \text{right subtree}$$

```
15

            /    \
         [6]      [18]
        /   \     /   \
      [3]  [7]  [17] [20]
      / \    \
    [2] [4] [13]
             /
           [9]
```

Property: For every node x:

- All keys in the **left** subtree < x.key
- All keys in the **right** subtree > x.key

This is the same principle as **binary search** – at each step, we eliminate half the search space.

BST Search

To search for key k in a BST:

```
TREE-SEARCH(x, k):
    if x == NIL or k == x.key
        return x
    if k < x.key
        return TREE-SEARCH(x.left, k)
    else
        return TREE-SEARCH(x.right, k)
```

Example: Search for 13

```
            [15]          15: 13 < 15, go left
           /    \
        [6]      [18]     6: 13 > 6, go right
       /   \
     [3]  [7]             7: 13 > 7, go right
            \
           [13]  ⟵ Found!
            /
          [9]
```

At each level, we go either left or right – **one comparison per level**.

BST Insertion

To insert key k, search for it. When we reach NIL, insert there:

```
TREE-INSERT(T, z):
    y = NIL
    x = T.root
    while x ≠ NIL
        y = x
        if z.key < x.key
            x = x.left
        else
            x = x.right
    z.parent = y
    if y == NIL
        T.root = z          // Tree was empty
    else if z.key < y.key
        y.left = z
    else
        y.right = z
```

Insertion follows the same path as an unsuccessful search, then attaches the new node as a **leaf**.

BST Search/Insert – Average Case

If data is inserted in **random order**, the tree height is approximately log n:

$$T(n) = T(n/2) + O(1)$$

- **T(n/2)**: At each node, we recurse into one subtree (roughly half the nodes)
- **O(1)**: One comparison at the current node

By the Master Theorem: a = 1, b = 2, f(n) = O(1)

$$T(n) = \Theta(\log n)$$

Average-case search and insert are both **O(log n)** – the same as binary search on a sorted array.

BST Worst Case – The Problem

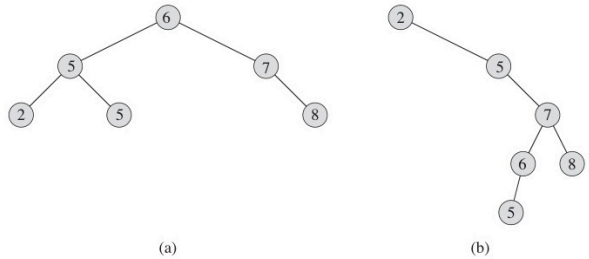
What if data is inserted in **sorted order**?

```
Insert: 1, 2, 3, 4, 5

    [1]
      \
      [2]
        \
        [3]
          \
          [4]
            \
            [5]
```

The tree degenerates into a **linked list**!

$$T(n) = T(n-1) + O(1) \implies T(n) = O(n)$$



Case	Height	Search/Insert
Average (random)	O(log n)	O(log n)

BST Deletion

Deletion is more complex than search or insert. Three cases:

Case 1: Node has **no children** – simply remove it.

```
5][5

  /   \   ⇒   /   \
[3]   [7]    [3]   [7]
  \
  [4]  ⟵ delete 4
```

Case 2: Node has **one child** – replace node with its child.

```
5][5

  /   \   ⇒   /   \
[3]   [7]    [4]   [7]
  \
  [4]  ⟵ delete 3, replace with 4
```

BST Deletion – Two Children

Case 3: Node has **two children** – find the **in-order successor**.

The **in-order successor** of node D is the smallest node in D's right subtree.

```
Delete 6:
         [15]                    [15]
        /    \                  /    \
     [6]      [18]          [7]      [18]
    /   \     /   \    ⇒   /  \     /   \
  [3]  [7]  [17] [20]    [3] [13]  [17] [20]
  / \    \                / \  /
[2] [4] [13]            [2][4][9]
         /
       [9]

Step 1: Find in-order successor of 6 ⇒ 7 (smallest in right subtree)
Step 2: Replace 6's key with 7
Step 3: Delete original node 7 (has at most one child)
```

This preserves the BST property because the successor is the next larger value.

BST Deletion – Complexity

Operation	Cost
Find the node to delete	O(h)
Find in-order successor	O(h)
Perform the deletion	O(1)

Total: O(h), where h is the tree height.

Case	Height h	Deletion
Average	O(log n)	**O(log n)**
Worst	O(n)	**O(n)**

Summary of all BST operations:

Part 2. Advanced Trees

Red-Black Trees and B-Trees

Red-Black Tree (RBT) – Motivation

BST worst case is O(n) – can we **guarantee** O(log n)?

Yes. A Red-Black Tree is a BST with one extra bit per node: a **color** (red or black).

The coloring rules ensure that the tree stays **approximately balanced**, so:

$$h \leq 2\log_2(n+1) = O(\log n)$$

All operations (search, insert, delete) are **guaranteed O(log n)** in the worst case.

Red-Black Tree — Five Properties

Every Red-Black Tree must satisfy these five properties:

#	Property
1	Every node is either **red** or **black**
2	The **root** is black
3	Every **leaf (NIL)** is black
4	If a node is **red**, both its children are **black** (no two consecutive reds)
5	For each node, all paths from that node to descendant leaves contain the **same number of black nodes**

7:B

```
          /     \
      [3:R]     [18:R]
      /   \     /    \
   [1:B] [5:B] [10:B] [22:B]
```

Why RBT Guarantees O(log n)

Property 5 (equal black-height on all paths) is the key:

- Let bh(x) = **black-height** = number of black nodes on any path from x to a leaf
- A subtree rooted at x has at least **2^bh(x) - 1** internal nodes

Property 4 (no consecutive reds) means:

- On any path, at least **half** the nodes are black
- Therefore: bh(root) >= h/2

Combining:

$$n \geq 2^{h/2} - 1 \implies h \leq 2\log_2(n+1)$$

The longest path (alternating red-black) is at most **twice** the shortest path (all black).

RBT Insertion – Overview

When inserting a new node:

1. Insert as in a normal BST
2. **Color the new node red** (to avoid violating Property 5)
3. Fix any violations – specifically, **Property 4** (Double Red)

```
Insert 4 into:

    [7:B]              [7:B]
   /     \            /     \
 [3:B]  [18:B]  ⇒  [3:B]  [18:B]
                       \
                      [4:R]  ⟵ New node, always red

No violation here. But what if the parent is also red?
```

Double Red Problem: New node is red AND its parent is red => Property 4 violated!

RBT – Double Red: Terminology

When Double Red occurs, we label the nodes:

```
      [G]          G = Grandparent
     /   \
   [P]   [U]      P = Parent,  U = Uncle
   /
 [N]               N = New node (just inserted)
```

Two cases based on the **uncle's color**:

Uncle's Color	Strategy
Black	**Restructuring** (rotation)
Red	**Recoloring** (color flip)

Case 1: Restructuring (Uncle is Black)

When U is black, we **restructure** (rotate) N, P, G:

Step 1: Sort N, P, G by key value

Step 2: The **median** becomes the new parent; the other two become children

Step 3: New parent becomes **black**; both children become **red**

```
Before (G=7, P=3, N=5):       After restructuring:

      [7:B]                        [5:B]
     /     \                      /     \
   [3:R]  [8:B]    ⇒         [3:R]   [7:R]
      \                            /
     [5:R]                       [8:B]

Sorted: 3, 5, 7
Median: 5 ⇒ becomes parent (black)
Others: 3, 7 ⇒ become children (red)
```

Restructuring is a **local** operation – it does NOT propagate upward. Done in **O(1)**.

Case 2: Recoloring (Uncle is Red)

When U is red, we **recolor**:

Step 1: P and U become **black**

Step 2: G becomes **red**

Step 3: If G is root, make it black. Otherwise, check if G causes a new Double Red.

```
Before:                        After recoloring:

      [7:B]                        [7:R] ← may cause
     /     \                      /     \    new Double Red!
   [3:R]  [8:R]    ⇒          [3:B]   [8:B]
   /                          /
 [1:R]                      [1:R]

P=3 and U=8 become black.
G=7 becomes red.
If 7 is root ⇒ force it black.
If 7's parent is also red ⇒ repeat fix-up upward!
```

Recoloring may **propagate** up the tree, but at most O(log n) times.

RBT – Restructuring Example (Step by Step)

Insert sequence: 7, 3, 8, 1, 5

```
Step 1: Insert 7        Step 2: Insert 3       Step 3: Insert 8
   [7:B]                  [7:B]                  [7:B]
                          /                      /     \
                        [3:R]                  [3:R]   [8:R]

Step 4: Insert 1 ⇒ Double Red! (N=1, P=3, U=8)
Uncle 8 is RED ⇒ Recoloring

      [7:B]                   [7:B]
     /     \       ⇒         /     \
   [3:R]  [8:R]           [3:B]  [8:B]
   /                       /
 [1:R]                   [1:R]

Step 5: Insert 5 ⇒ Double Red! (N=5, P=3, U=8)
Uncle 8 is BLACK? No, recheck: P=3, G=7, U=8 is black now.
⇒ Restructuring: sort {1? no, N=5, P=3, G=7}
Actually P=3 is parent, but let's trace carefully:

      [7:B]          N=5, P=3, G=7, U=8(B)
     /     \         Uncle is black ⇒ Restructure
   [3:B]  [8:B]     Sort N,P,G: 3, 5, 7
      \              Median=5 ⇒ new parent
```

RBT – Operation Complexities

Operation	Complexity	Details
Search	O(log n)	Same as BST (tree height is O(log n))
Insert	O(log n)	BST insert O(log n) + fix-up O(log n)
Delete	O(log n)	BST delete O(log n) + fix-up O(log n)

The fix-up after insertion or deletion involves at most **O(log n)** recolorings and at most **2 rotations**.

Key takeaway: RBT pays a small constant overhead for balancing, but **guarantees** O(log n) for all operations – unlike plain BST which can degrade to O(n).

	BST (average)	BST (worst)	RBT (worst)
Search	O(log n)	O(n)	**O(log n)**

B-Tree – Motivation: The Disk I/O Problem

When data is too large for main memory, it lives on **disk**.

The bottleneck: Disk access is enormously slower than CPU operations.

```
Access Times:
  CPU register     ~0.3 ns
  Main memory      ~100 ns
  SSD              ~50 us      (500x slower than memory)
  HDD              ~5 ms       (50,000x slower than memory)
```

A Red-Black Tree with n = 10^9 nodes has height ~30.

- **30 disk accesses** = potentially 30 x 5ms = **150ms** on HDD
- "1 second of CPU ≈ 10 days of disk I/O" (in relative terms)

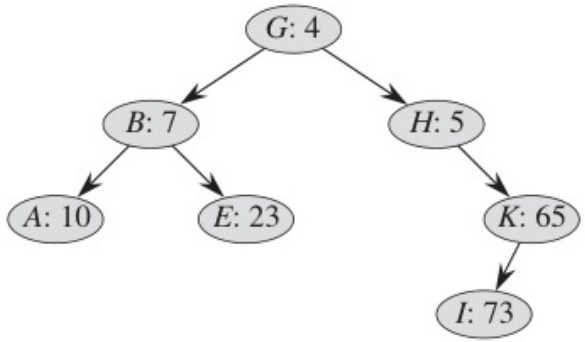
Goal: Minimize the number of **disk accesses** (tree height).

B-Tree — The Idea

Solution: Make each node hold **many keys** so the tree is **wide and shallow**.

```
BST (tall, narrow):         B-Tree (short, wide):

     [50]                  [20|40|60|80]
    /    \                / |  |  |  \
  [25]  [75]            [ .. ][ .. ][ .. ][ .. ][ .. ]
 / \    / \
......                   Height ~3 for 10^9 nodes!
Height ~30
```



Key design decisions:

- One node = one **disk block** (typically 4KB)
- One disk read loads **hundreds of keys** at once
- Internal search within a node uses fast **CPU operations**
- A B-Tree of order 1000 with height 3 can store over **10^9 keys**

B-Tree — Search

To search for key k:

1. Start at the **root**
2. Scan the keys in the current node
3. If k is found, **return** success
4. If k falls between two keys, follow the corresponding **child pointer**
5. Repeat until found or a **leaf** is reached (search failure)

```
Search for key 42 in a B-Tree (order 5):

  Root: [20 | 40 | 60 | 80]
         |    |    |    |
         v    v    v    v
        ...  ...  ...  ...

  42 > 40 and 42 < 60 ⇒ follow pointer between 40 and 60

  Child: [41 | 42 | 45 | 50]
                ^
                Found! (1 disk read for root + 1 disk read for child = 2 total)
```

B-Tree — Insertion (No Split)

Case 1: The target leaf has room — simply insert in sorted order.

```
Insert 25 into this B-Tree (max 4 keys per node):

  Root: [10 | 20 | 40 | 60]
              |
              v
  Leaf: [21 | 30 | 35]        ⟵ has room (3 keys, max is 4)

  After insert:
  Leaf: [21 | 25 | 30 | 35]   ⟵ just place 25 in sorted position
```

No structural change needed. One search + one write.

B-Tree — Insertion (With Split)

Case 2: The target leaf is **full** — we must **split** and promote.

```
Insert 28 into this leaf (max 4 keys per node):

  Parent: [ ... | 20 | 40 | ... ]
                 |
                 v
  Leaf: [21 | 25 | 30 | 35]    ⟵ FULL! Cannot insert.

Step 1: Insert 28, creating temporary overflow:
  Temp: [21 | 25 | 28 | 30 | 35]

Step 2: Split at median (28):
  Left:  [21 | 25]
  Right: [30 | 35]

Step 3: Promote median (28) to parent:
  Parent: [ ... | 20 | 28 | 40 | ... ]
                 |    |
                 v    v
        [21 | 25]    [30 | 35]
```

If the parent is also full, the split **propagates upward**. In the extreme case, the root splits and the tree grows one level taller.

B-Tree – Deletion Overview

Deletion is the most complex B-Tree operation. Main cases:

Case 1: Key is in a **leaf** node

- 1.1: Node has more than minimum keys => simply delete
- 1.2: Borrow from a sibling (sibling has extra keys) => rotate through parent
- 1.3: Merge with sibling (both at minimum) => merge and pull down parent key
- 1.4: Merge causes parent underflow => recurse upward (restructure)

Case 2: Key is in an **internal** node

- Replace with **in-order predecessor** (largest in left child) or **in-order successor** (smallest in right child)
- Then delete from the leaf => reduces to Case 1

Case 3: Key is internal, node and children all at minimum

- Merge children, pull down parent key, restructure
- May propagate splits or merges upward

B-Tree Deletion — Leaf Cases (Illustrated)

Case 1.1: Simply remove (node has extra keys)

```
Delete 30:  [10 | 20 | 30 | 40]  ⇒  [10 | 20 | 40]   ✓
```

Case 1.2: Borrow from sibling via parent rotation

```
Delete 20:
  Parent: [ ... | 25 | ... ]        Parent: [ ... | 30 | ... ]
            |    |                            |    |
          [20] [30 | 40]   ⇒            [25] [40]

Borrow 30 from right sibling; 25 rotates down, 30 rotates up.
```

Case 1.3: Merge with sibling

```
Delete 20:
  Parent: [ ... | 25 | ... ]       Parent: [ ... | ... ]
            |    |                           |
          [20] [30]        ⇒         [25 | 30]

Merge: pull 25 down, combine with sibling.
```

BST vs RBT vs B-Tree – Comparison

Property	BST	Red-Black Tree	B-Tree
Type	Binary	Binary (balanced)	M-way (balanced)
Height	O(log n) avg, O(n) worst	O(log n) guaranteed	O(log_M n) guaranteed
Search	O(log n) avg, O(n) worst	O(log n)	O(log_M n)
Insert	O(log n) avg, O(n) worst	O(log n)	O(log_M n)
Delete	O(log n) avg, O(n) worst	O(log n)	O(log_M n)

Summary

1. **Trees are recursive** – traversal is Theta(n), and many tree algorithms use divide-and-conquer
2. **BST** provides O(log n) average-case search, insert, and delete, but degrades to O(n) with sorted input
3. **Red-Black Trees** add 1 bit of color per node and enforce 5 properties to guarantee:
   - Height <= 2 log(n+1) => all operations O(log n) worst case
   - Double Red fixed by **Restructuring** (uncle black) or **Recoloring** (uncle red)
4. **B-Trees** are designed for disk-based storage:
   - Wide nodes (hundreds of keys) => very shallow trees
   - Minimize disk I/O: height ~3 for billions of keys
   - Used in databases (MySQL, PostgreSQL) and file systems (NTFS, ext4)

Q & A

codingchild@korea.ac.kr